

AWS Load-Balanced Web App with a CI/CD Pipeline

Stack: VPC, VPC Peering, ALB, EC2, Ansible, Artifactory, Jenkins, GitHub **Region:** us-east-1 (N. Virginia) | **OS:** Amazon Linux 2023 on all servers **Author:** Param

Placeholders in `<ANGLE_BRACKETS>` are values you replace. **Never type the `<` `>` characters.**

1. Project overview

What it does

The project hosts a web page on two EC2 servers behind an Application Load Balancer. A CI/CD pipeline publishes changes automatically. A second private VPC is connected through VPC Peering.

User path: Browser → (Route 53, optional) → Internet → Internet Gateway → ALB → one of two EC2 servers → response. Route 53 only answers DNS questions. It never carries web traffic.

Delivery path: Git push → Jenkins builds a package → Artifactory stores it → Ansible deploys it to both servers.

Network layout

Item	Value
Software VPC	software-vpc , 10.0.0.0/24
public-subnet-a	10.0.0.0/26 , us-east-1a (app server 1, ALB)
public-subnet-b	10.0.0.64/26 , us-east-1b (app server 2, ALB)
tools-subnet	10.0.0.128/26 , us-east-1a (Jenkins, Artifactory, Ansible)
Customer VPC	customer-vpc , 11.0.0.0/24
customer-private-subnet	11.0.0.0/26 , no internet route

Peering

software-customer-pcx

Servers

Name	VPC / subnet	Type	Security group
app-server-1	software / public-a	t3.micro	web-sg
app-server-2	software / public-b	t3.micro	web-sg
tools-server	software / tools-subnet	t3.large, 30 GB disk	tools-sg
customer-ec2 (11.0.0.10)	customer / private	t3.micro	customer-sg

Current status

Part	Status
VPCs, subnets, IGW, routes, security groups	Built
App servers with nginx (Ansible)	Verified
Artifactory + PostgreSQL	Verified (upload works)
Jenkins pipeline	Verified (Finished: SUCCESS)
ALB with both targets Healthy	Verified (Figures 1, 3, 5)
VPC peering and ping 11.0.0.10	Not complete yet: the resource maps (Figures 6, 7) show no peering connection
Version 2 auto-update demo	Pending (Figure 3 still shows Version 1)
Route 53 / HTTPS	Not implemented (not free-tier)

Evidence (screenshots)

Account IDs and public IP addresses are blacked out.

Figure 1. First deployment. The ALB DNS name serves the page that Ansible deployed from Artifactory.



Figure 2. Artifactory users. The `deploy` user has admin rights; without them uploads return 403.

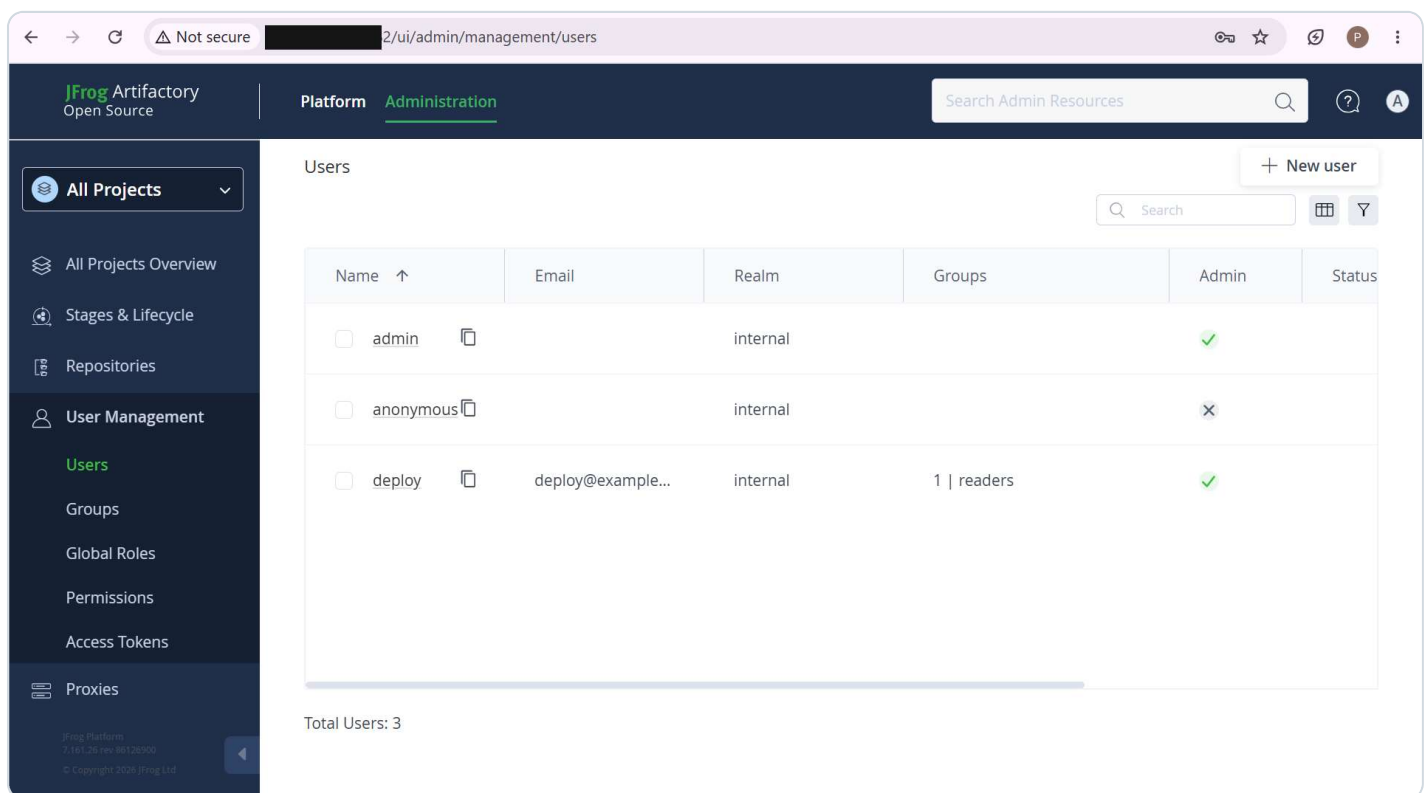


Figure 3. Page deployed by Jenkins. The same ALB address now shows the page built and deployed by the pipeline.



Figure 4. Jenkins pipeline. Build #2 failed because app servers could not reach Artifactory on port 8082; after the security group fix, builds #3 and #4 succeeded.

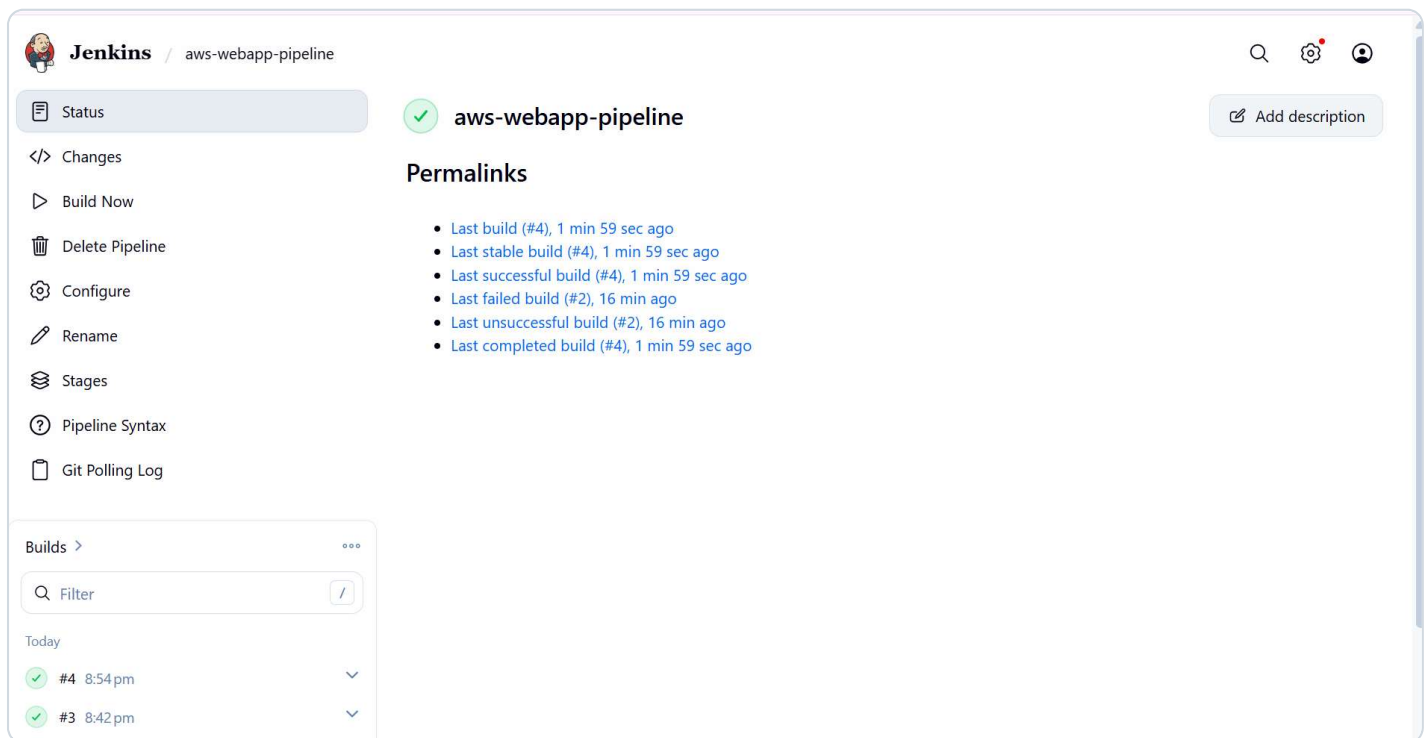
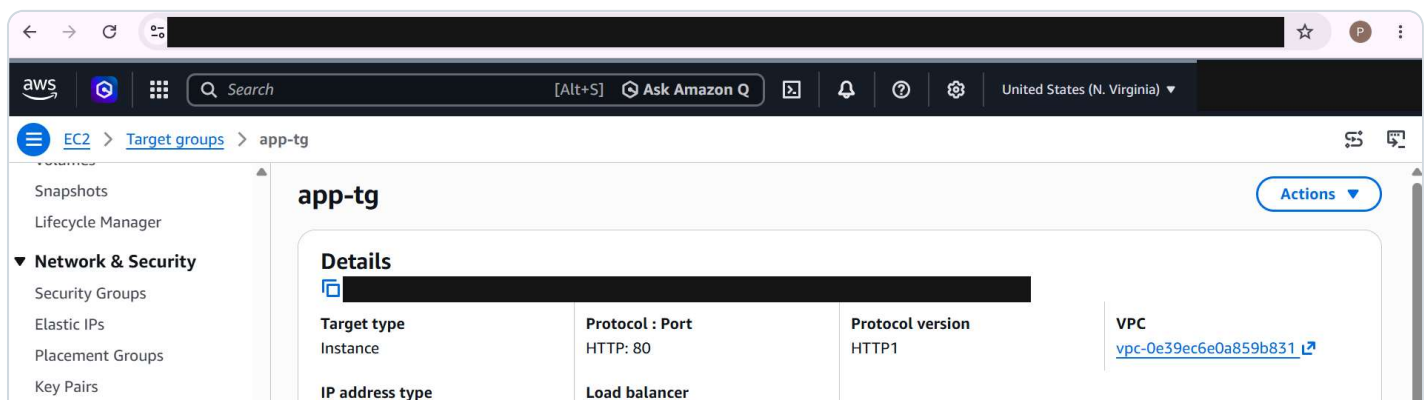


Figure 5. Target group. Both servers behind app-alb report Healthy.



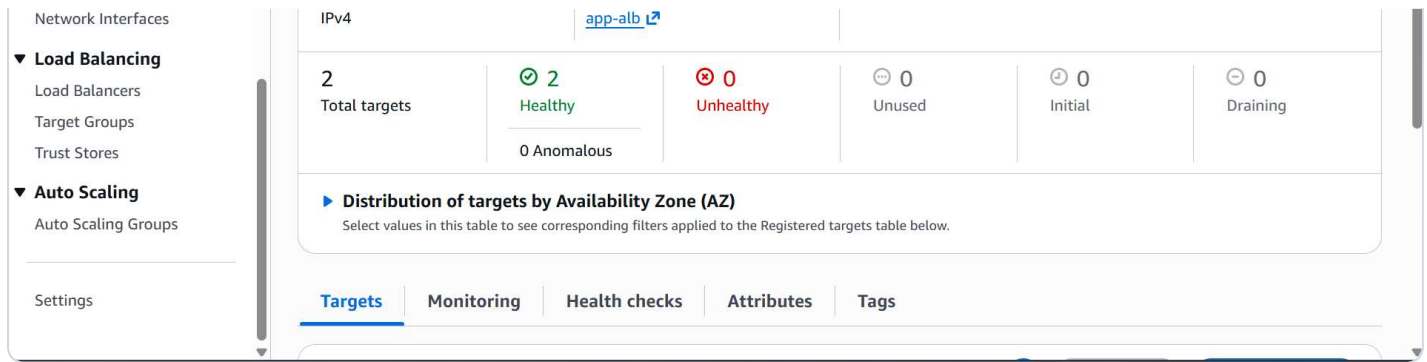


Figure 6. Software VPC resource map. software-vpc (10.0.0.0/24) with three subnets across two Availability Zones, the public route table and the internet gateway.

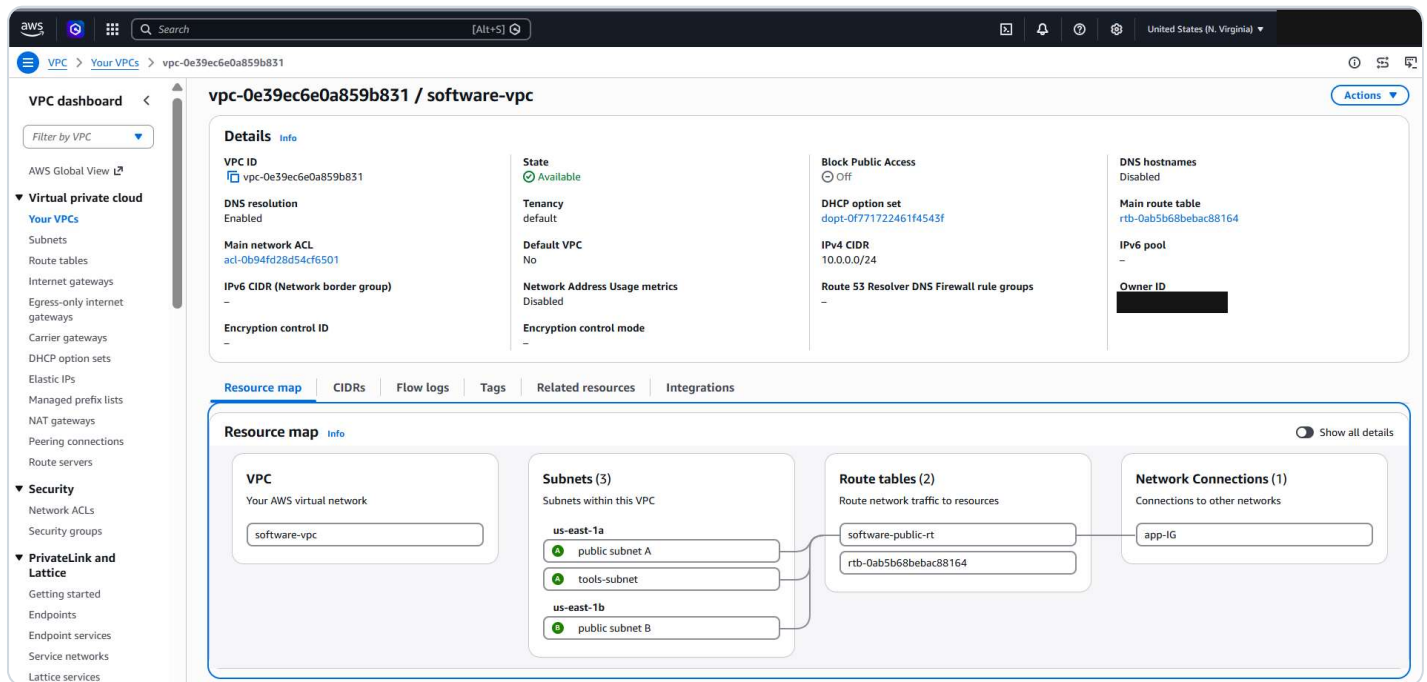
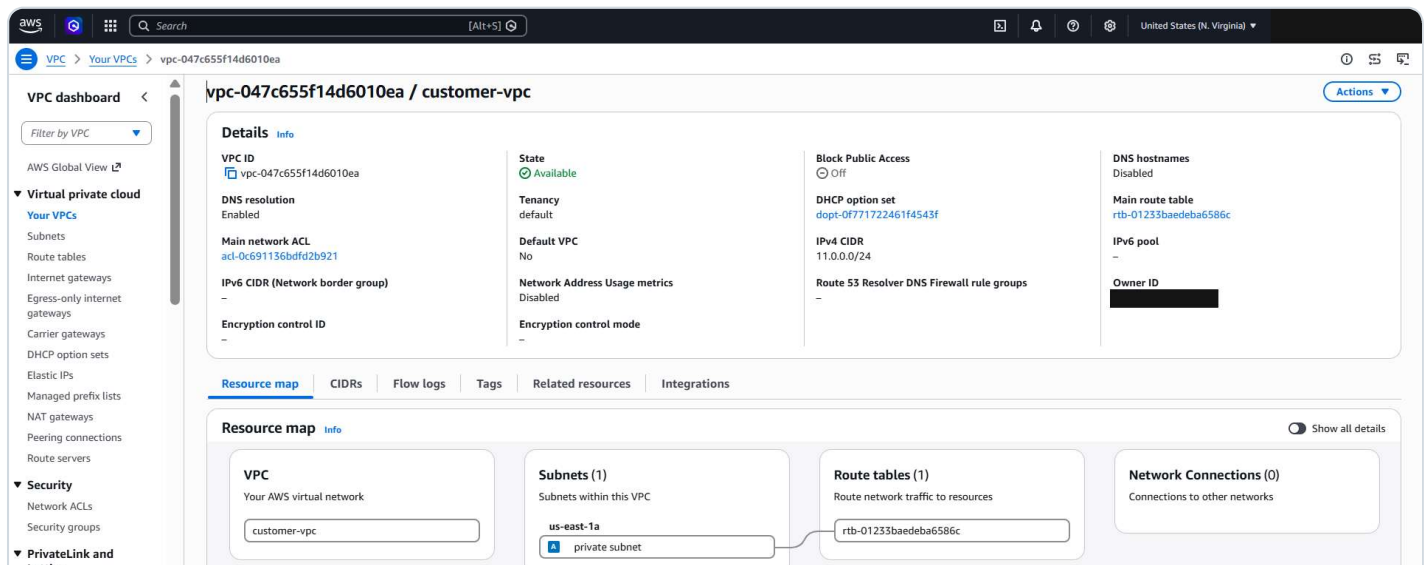


Figure 7. Customer VPC resource map. customer-vpc (11.0.0.0/24) with one private subnet and no internet connection. "Network Connections (0)" shows the peering link is not set up yet.



2. What each tool does

Tool	Role	Why it is here
VPC	Your private network in AWS	Isolates the project; you choose the IP ranges
Subnet	A slice of the VPC in one Availability Zone	Two AZs give resilience
Internet Gateway	Door between the VPC and the internet	Without it nothing is reachable from outside
Route table	Signposts for traffic	0.0.0.0/0 → IGW is what makes a subnet "public"
Security group	Stateful firewall per resource	Each server only trusts who needs access
VPC Peering	Private link between two VPCs	Lets the customer server reach the app network without the internet
EC2	Virtual servers	Run nginx and the tools
ALB	Load balancer	One entry point, health checks, spreads traffic
Target group	List of servers behind the ALB plus the health check	The ALB only knows the target group
Route 53	DNS	Maps a domain name to the ALB (optional here)
Docker	Runs containers	Runs Artifactory and PostgreSQL
Artifactory OSS	Artifact repository	Stores versioned packages (webapp-1 , webapp-latest)
PostgreSQL	Database	Artifactory OSS cannot start without it
Ansible	Configuration and deployment	Same result on every server, repeatable

	over SSH	
Jenkins	CI/CD automation	Build, upload and deploy on every Git change
GitHub	Source control	Holds the code and Jenkinsfile

3. The practical, step by step

Phase 0: Preparation

1. Sign in as an **IAM user**, not root. Switch the region to **N. Virginia (us-east-1)**.
2. **EC2** → **Key Pairs** → **Create key pair**: `project-key` , RSA, `.pem` . **It downloads only once.**
Save it somewhere you will remember (for example `D:\project-key.pem`).
3. Set a **zero-spend budget alert** (Billing → Budgets).

Phase 1: Software VPC

1. **VPC** → **Create VPC** → **VPC only**: `software-vpc` , `10.0.0.0/24` . Enable **DNS hostnames**.
2. Create **three** subnets (table in section 1). Enable **auto-assign public IPv4** on each.
3. Create an **Internet Gateway** `app-IG` and **attach** it to the VPC.
4. Create route table `software-public-rt` , add `0.0.0.0/0` → `app-IG` , and **associate all three subnets**. A new subnet uses the main table until you do this.

Phase 2: Customer VPC

Create `customer-vpc` (`11.0.0.0/24`) and `customer-private-subnet` (`11.0.0.0/26`). No IGW.
The CIDRs of two peered VPCs must never overlap.

Phase 3: Security groups (create these before the servers)

Group	VPC	Inbound rules
alb-sg	software	HTTP 80 and HTTPS 443 from <code>0.0.0.0/0</code>
tools-sg	software	SSH 22, TCP 8080, TCP 8082 from your IP ; TCP 8082 from <code>web-sg</code> (app servers download here)
web-sg	software	HTTP 80 from <code>alb-sg</code> ; SSH 22 from <code>tools-sg</code> ; HTTP 80 and ICMP from <code>11.0.0.0/24</code>
customer-vpc-sg	customer-vpc	SSH, ICMP, HTTP from <code>10.0.0.0/24</code>

customer- sg	customer	SSH, ICMP, HTTP FROM 10.0.0.0/24
-----------------	----------	----------------------------------

Leave outbound at the default (all traffic). Jenkins and Artifactory have no preset in the Type dropdown, so choose **Custom TCP** and type the port.

Phase 4: VPC Peering

1. **VPC** → **Peering connections** → **Create** (requester software-vpc , acceptor customer-vpc), then **Accept request**.
2. Add routes on **both** sides:
3. software-public-rt : 11.0.0.0/24 → pcx-...
4. Customer VPC route table (the one the private subnet uses): 10.0.0.0/24 → pcx-...
5. Test later from the tools server: `ping -c 3 11.0.0.10` .

Phase 5: Servers, target group, ALB

1. Launch each server **one by one** (the wizard's "2 instances" puts both in one subnet). Select **existing** security group each time.
2. **Target group** app-tg : type Instances, HTTP 80, VPC software-vpc , health check path / . Choose both servers, then click **Include as pending below**.
3. **ALB** app-alb : internet-facing, mapped to **both** public subnets, security group alb-sg , listener HTTP 80 → app-tg .
4. Targets stay **Unhealthy** (and the ALB returns 503) until nginx is installed in Phase 7.

Phase 6: Tools server

Launch tools-server as **t3.large with 30 GB storage** in tools-subnet , then connect (section 4).

```
sudo dnf update -y
sudo dnf install -y git docker ansible-core fontconfig
sudo systemctl enable --now docker
sudo usermod -aG docker ec2-user      # then log out and back in
```

Artifactory needs PostgreSQL. One container is not enough.

```
docker network create jfrog-net
docker run -d --name postgres --network jfrog-net --restart unless-stopped \
  -e POSTGRES_DB=artifactory -e POSTGRES_USER=artifactory \
  -e POSTGRES_PASSWORD=<DB_PASSWORD> -v pgdata:/var/lib/postgresql/data postgres:15
```

```

sudo mkdir -p /opt/artifactory/var && sudo chown -R 1030:1030 /opt/artifactory/var
docker run -d --name artifactory --network jfrog-net --restart unless-stopped \
  -p 8081:8081 -p 8082:8082 \
  -e JF_SHARED_DATABASE_TYPE=postgresql \
  -e JF_SHARED_DATABASE_DRIVER=org.postgresql.Driver \
  -e JF_SHARED_DATABASE_URL=jdbc:postgresql://postgres:5432/artifactory \
  -e JF_SHARED_DATABASE_USERNAME=artifactory \
  -e JF_SHARED_DATABASE_PASSWORD=<DB_PASSWORD> \
  -v /opt/artifactory/var:/var/opt/jfrog/artifactory \
  releases-docker.jfrog.io/jfrog/artifactory-oss:latest

until curl -sf http://localhost:8082/artifactory/api/system/ping; do echo waiting; sle

```

Open `http://<TOOLS_PUBLIC_IP>:8082` , log in `admin / password` , set a new password, skip base URL and proxy, and pick **Generic**. Create the **local Generic** repository `webapp-releases` . Create user `deploy` and tick **Platform Administrator** (otherwise uploads return 403). Test:

```

echo test > test.txt
curl -u 'deploy:<ART_PASSWORD>' -T test.txt http://localhost:8082/artifactory/webapp-r

```

Jenkins needs Java 21:

```

sudo dnf install -y java-21-amazon-corretto-headless
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkir
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
sudo dnf install -y jenkins
sudo systemctl enable --now jenkins
sudo cat /var/lib/jenkins/secrets/initialAdminPassword

```

Give Jenkins its own copy of the SSH key (it runs as the `jenkins` user):

```

sudo mkdir -p /var/lib/jenkins/.ssh
sudo cp ~/.ssh/project-key.pem /var/lib/jenkins/.ssh/
sudo chown -R jenkins:jenkins /var/lib/jenkins/.ssh

```

```
sudo chmod 700 /var/lib/jenkins/.ssh && sudo chmod 400 /var/lib/jenkins/.ssh/project-k
```

Phase 7: Ansible deployment

~/deploy/inventory.ini :

```
[web]
app1 ansible_host=<APP1_PRIVATE_IP>
app2 ansible_host=<APP2_PRIVATE_IP>

[web:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=/home/ec2-user/.ssh/project-key.pem
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
ansible_python_interpreter=/usr/bin/python3.9
```

~/deploy/deploy.yml installs nginx and tar , starts nginx, downloads webapp-latest.tar.gz from `http://<TOOLS_PRIVATE_IP>:8082/artifactory/webapp-releases/` with `get_url` (basic auth), unpacks it into `/usr/share/nginx/html` , and writes `whoami.html` containing `{{ inventory_hostname }}` .

```
ansible web -i inventory.ini -m ping
ansible-playbook -i inventory.ini deploy.yml -e 'artifactory_user=deploy' -e 'artifact
```

Check the servers from inside (the tools server cannot `curl` them directly, by design):

```
ansible web -i inventory.ini -m shell -a "curl -s http://localhost"
```

Phase 8: Jenkins pipeline

1. GitHub repo `aws-devops-project` (public, never commit passwords or the `.pem`) containing `index.html` , `inventory.ini` (key path `/var/lib/jenkins/.ssh/project-key.pem`), `deploy.yml` , `Jenkinsfile` .
2. Jenkinsfile stages: **Build** (`tar -czf webapp-latest.tar.gz index.html`), **Upload to Artifactory** (`webapp-latest` plus a numbered copy `webapp-${BUILD_NUMBER}`), **Deploy** (`ansible-playbook`). Trigger: `pollSCM('H/2 * * * *')` . Credentials come from

withCredentials .

3. **Manage Jenkins** → **Credentials** → **System** → **Global** → **Add Credentials**: Username with password, ID artifactory-creds . (Not "Manage Jenkins → System", which is global settings.)
4. **New Item** → **Pipeline** → **Pipeline script from SCM** → **Git**, repo URL, branch */main , script path Jenkinsfile . Click **Build Now**.
5. Demo: change Version 1 to Version 2 on GitHub, wait about two minutes, reload the ALB URL.

Phase 9: Route 53 (optional)

A hosted zone costs about \$0.50 a month and is not free-tier. Alias records to an ALB are free. Without it, use the ALB's DNS name. The record would be: type A, Alias, target app-alb .

4. Command cheat sheet

On your laptop (PowerShell, prompt PS C:\...>):

```
icacls D:\project-key.pem /inheritance:r /grant:r "$($env:USERNAME):(R)"
ssh -i D:\project-key.pem ec2-user@<TOOLS_PUBLIC_IP>
scp -i D:\project-key.pem D:\project-key.pem ec2-user@<TOOLS_PUBLIC_IP>:~/.ssh/
# tunnel: reach Jenkins/Artifactory without opening ports to the internet
ssh -i D:\project-key.pem -L 8080:localhost:8080 -L 8082:localhost:8082 ec2-user@<TOOL
```

On the server (prompt [ec2-user@ip-... ~]\$):

```
docker ps                # containers running?
docker logs -f artifactory # Ctrl+C stops viewing only
sudo systemctl status jenkins # q to exit
free -h ; df -h /         # memory and disk
curl http://localhost:8082/artifactory/api/system/ping # prints OK
ALB=<ALB_DNS_NAME>
for i in {1..10}; do curl -s http://$ALB/whoami.html; done # app1/app2 alternate
```

Which window am I in? PS C:\...> is your laptop (use scp , ssh). [ec2-user@ip-...]\$ is the server (use docker , chmod , ansible).

5. Mistakes made and how not to repeat them

Typing and environment

Mistake	Lesson
Typed <code><new-public-ip></code> and <code><alb-dns-name></code> literally	Replace placeholders including the brackets
Typed <code>curl http://10.0.0.61</code> in the browser bar	<code>curl</code> is a terminal command; private IPs only work inside the VPC
Ran Linux <code>chmod</code> in PowerShell	Use <code>icacls</code> on Windows
Could not find <code>project-key.pem</code>	Save the key in a known folder; it downloads only once
Pasted commands into the wrong window	Check the prompt before running anything
Typed <code>test.txtt</code>	Re-read commands before running
Used <code>YOUR-PASSWORD</code> as the real password	Placeholders must be replaced

AWS and networking

Mistake	Lesson
Used the browser Connect button with an SSH rule limited to My IP	That connection comes from AWS, not your laptop. Use SSH from your PC
My IP changes (Wi-Fi vs mobile data) and every page times out	Refresh the rule; check https://checkip.amazonaws.com . Existing SSH sessions keep working, which hides the problem
Re-selecting My IP overwrote the rule the app servers needed on port 8082 (Ansible download timed out, <code>rc=28</code>)	Keep separate rules for people and for servers; use <code>web-sg</code> as the source for servers
New subnet not associated with the public route table	Always associate it, or it has no internet

Left VPC peering until last and never completed it	Do it before the tools; then both resource maps show a peering connection
Peering "accepted" but no ping	Add routes on both sides plus security group rules
Confused the route table Main column with the route's Propagated column	Main = No is normal for tables you created
Counted four subnets in the guide	The design has three Software subnets plus one Customer subnet
Diagram errors: 11.0.0.10 inside the Software VPC, 10.0.0.20 is not in subnet B, title says "EC2 + EC2"	Fix the diagram before publishing it

Servers and tools

Mistake	Lesson
Launched the tools server as t3.micro (1 GB)	Artifactory and Jenkins need 4 GB or more; use t3.large
Disk was 8 GB, not 30 GB	Set storage in the launch wizard; if late, modify the volume, then <code>sudo growpart /dev/nvme0n1 1</code> and <code>sudo xfs_growfs -d /</code>
<code>docker ps</code> gave "permission denied"	Group changes need a fresh login (or <code>newgrp docker</code>)
Artifactory crashed: "Cannot start the application with a database other than PostgreSQL"	OSS needs PostgreSQL. Read the first real error in the logs, not the retry spam
Container name already in use	<code>docker rm -f <name></code> before re-running
REST call for users failed: "available only in Artifactory Pro"	The OSS edition limits the API; use the UI
<code>deploy</code> user got 403 on upload, 401 on a wrong password	401 = unknown credentials; 403 = known user without permission. Tick Platform Administrator
Wizard hung and the new admin password did not save	Verify credentials with <code>curl</code> before moving on

Jenkins failed immediately on Java 17	Install Java 21 and run <code>sudo systemctl reset-failed jenkins</code>
Opened Manage Jenkins → System instead of Credentials	Credentials are under Security
Jenkins used the wrong key path	The <code>jenkins</code> user reads <code>/var/lib/jenkins/.ssh/</code> , not <code>/home/ec2-user/</code>
Expected <code>curl</code> to the app servers to work from the tools server	<code>web-sg</code> allows port 80 only from the ALB. Test through the ALB or with <code>ansible ... -m shell</code>

Process and security

Mistake	Lesson
Worked until 2 am	Stop instances and continue fresh; errors multiply when tired
Passwords shared in chat and screenshots	Treat them as leaked: change them (Artifactory and the Jenkins credential)
Opening ports to <code>0.0.0.0/0</code> was tempting	Keep admin ports limited, or use the SSH tunnel
Resources left running	The ALB and t3.large bill hourly; delete or stop after screenshots

6. Troubleshooting guide

Symptom	Likely cause	Fix
Page "took too long to respond"	Your IP changed, wrong server IP, or rule missing	Refresh My IP ; confirm current public IP; check the port rule
ERR_CONNECTION_REFUSED	Service not running	<code>docker ps</code> , <code>systemctl status jenkins</code>

ERR_SSL_PROTOCOL_ERROR	Typed https://	Use http://
ALB shows 503	No healthy targets	Wait; check web-sg allows port 80 from alb-sg
Targets Unhealthy, "timed out"	Security group or subnet mapping	web-sg source must be alb-sg
Ansible UNREACHABLE	SSH rule or key	web-sg port 22 from tools-sg ; check the key path and mode 400
Ansible download rc=28 or timeout	Port 8082 blocked from app servers	Add TCP 8082 from web-sg to tools-sg
Jenkins build: Couldn't find any revision	Branch name	Use */master if your branch is master
Jenkins: curl ... 401	Wrong credential	Recheck artifactory-creds
Version 2 not showing	No new build, wrong branch, or cached page	Click Build Now ; confirm GitHub shows Version 2; press Ctrl+F5; run <code>ansible web -i inventory.ini -m shell -a "cat /usr/share/nginx/html/index.html"</code>

7. Cost and cleanup

Delete in this order: **ALB**, target group, **stop or terminate EC2 instances** (the t3.large first), peering connection, any Route 53 hosted zone, any Elastic IP not attached to a running instance, then the VPCs. Subnets, route tables and security groups are free. Stopping an instance changes its public IP unless you attach an Elastic IP.

8. Security notes

- No passwords or `.pem` files in Git.
- Rotate any password that appeared in chats or screenshots.
- Keep SSH, 8080 and 8082 limited to your IP, or use the SSH tunnel.
- Production would use IAM roles and SSM Session Manager instead of copying a key, and private subnets for app servers.

9. Explaining it in an interview

I built a multi-AZ web application on AWS: two EC2 servers in separate subnets behind an Application Load Balancer, with a second VPC connected over peering. Jenkins watches GitHub, packages the app, stores a versioned artifact in Artifactory, and Ansible deploys it to both servers. Security groups restrict the web port to the load balancer and SSH to the tools server.

Resume bullets

- Designed and deployed a multi-AZ AWS architecture (VPC, ALB, EC2, VPC Peering) with least-privilege security groups.
- Built a CI/CD pipeline (GitHub, Jenkins, Artifactory, Ansible) that deploys to two servers automatically on every commit.
- Diagnosed network, firewall, memory and database startup issues (security group sources, PostgreSQL requirement, Java version).

9b. Future improvements

Move app servers to private subnets with a NAT gateway, add HTTPS with ACM and Route 53, replace polling with a GitHub webhook, rebuild the infrastructure with Terraform, and containerize the app.

